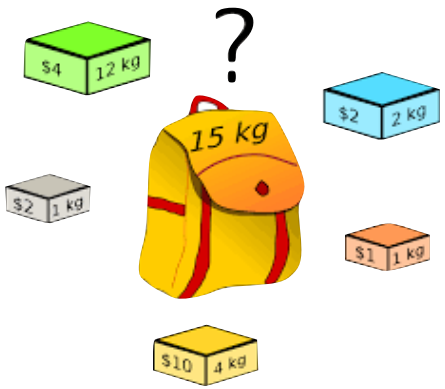
Análisis y Diseño de Algoritmos I

Valeria Bengolea
Departamento de Computación
Facultad de Ciencias Exactas, Físico-Químicas y Naturales
Universidad Nacional de Río Cuarto

Problema de la Mochila

El problema de la Mochila

El problema de la mochila es un problema clásico de optimización. Consideremos el caso en el que contamos con n items, cada uno con peso w_i y valor v_i. Dada una mochila de capacidad (peso máximo soportado) W, **encontrar el subconjunto de items que quepa en la mochila, y que maximice la suma de los valores del subconjunto.**



El Problema de la Mochila

Ejemplo: Supongamos que tenemos una mochila de capacidad W=16, y los siguientes items:

item	peso	valor
1	2	$20
2	5	$30
3	10	$50
4	5	$10

El Problema de la Mochila por Fuerza Bruta

Para resolverlo por fuerza bruta, consideremos todos los posibles subconjuntos de items:

item	peso	valor
1	2	$20
2	5	$30
3	10	$50
4	5	$10

Subconjunto	Peso total	Valor total
{1}	2	$20
{2}	5	$30
{3}	10	$50
{4}	5	$10
{1,2}	7	$50
{1,3}	12	$70
{1,4}	7	$30
{2,3}	15	$80
{2,4}	10	$40
{3,4}	15	$60
{1,2,3}	17	NO REALIZABLE
{1,2,4}	12	$60
{1,3,4}	17	NO REALIZABLE
{2,3,4}	20	NO REALIZABLE
{1,2,3,4}	22	NO REALIZABLE

Una Implementación en Fuerza Bruta

```
Algorithm knapsack(weights[0..n−1], values[0..n−1], W)
  powerSet ← { ∅ }
  maxValue ← 0

  for i ← 0 to n−1 do
    powerSetAux ← ∅
    for each subset ∈ powerSet do
      newSubset ← subset ∪ { i }

      if totalWeight(newSubset, weights) ≤ W then
        powerSetAux ← powerSetAux ∪ { newSubset }

        if totalValue(newSubset, values) > maxValue then
          maxValue ← totalValue(newSubset, values)
        end if
      end if
    end for
    powerSet ← powerSet ∪ powerSetAux
  end for
  return maxValue
```

Búsqueda Exhaustiva

Problema de la mochila

Consideremos una instancia del problema definida para los primeros i items $1 <= i <= n$ con pesos $w_1,...,w_i$, valores $v_1,...,v_i$ y capacidad de la mochila j: $1 \leq j \leq W$,

Asumimos knapsack(i,j) calcula el conjunto más valuable de llenar una mochila de capacidad j con los primeros i items.

Problema de la mochila

Podemos dividir todos los subconjuntos de los primeros i items que entran en la mochila de capacidad j en dos categorías:

Los que NO incluyen el i-ésimo item y los que sí lo incluyen:

Entre los que no lo incluyen: knapsack(i-1,j), es la solución óptima.

Entre los que sí lo incluyen, knapsack(i-1,j-w_i) + v_i es la solución optima.

Problema de la mochila

- El valor de una solución óptima entre todos los subconjuntos factibles de los primeros i items es el máximo de estos dos valores.
- Si el i-ésimo item no entra en la mochila, el valor del subconjunto óptimo seleccionado de los primeros i artículos es el mismo que el valor de un subconjunto óptimo seleccionado de los primeros i – 1 items.

$$F(0,\ j) = 0 \ \text{for}\ j \geq 0 \quad \text{and} \quad F(i,\ 0) = 0 \ \text{for}\ i \geq 0.$$

$$F(i,\ j) = \begin{cases} \max\{F(i-1,\ j),\ v_i + F(i-1,\ j-w_i)\} & \text{if } j - w_i \geq 0, \\ F(i-1,\ j) & \text{if } j - w_i < 0. \end{cases}$$

Problema de la mochila

```
Algoritmo knapSack(i, w)
  // Caso base: si no hay capacidad o no hay ítems
  Si (i = 0) O (w = 0) Entonces
    Retornar 0
  FinSi

  // Si el peso del ítem actual excede la capacidad restante
  Si (itemsWeight[i - 1] > w) Entonces
    Retornar knapSack(i-1, w)
  Sino
    // Incluir el ítem actual
    incluir ← itemsValue[i - 1] + knapSack(i-1, w - itemsWeight[i - 1])
    // Excluir el ítem actual
    excluir ← knapSack(i-1, w)
   Retornar max(incluir, excluir)
  FinSi
```

$$F(0,\, j) = 0 \text{ for } j \geq 0 \quad \text{and} \quad F(i,\, 0) = 0 \text{ for } i \geq 0.$$

$$F(i,\, j) = \begin{cases} \max\{F(i-1,\, j),\, v_i + F(i-1,\, j-w_i)\} & \text{if } j - w_i \geq 0, \\ F(i-1,\, j) & \text{if } j - w_i < 0. \end{cases}$$

Problema de la mochila
Programación Dinámica

$$F(0,\ j) = 0 \text{ for } j \geq 0 \quad \text{and} \quad F(i,\ 0) = 0 \text{ for } i \geq 0.$$

$$F(i,\ j) = \begin{cases} \max\{F(i-1,\ j),\ v_i + F(i-1,\ j-w_i)\} & \text{if } j - w_i \geq 0, \\ F(i-1,\ j) & \text{if } j - w_i < 0. \end{cases}$$

		0	$j-w_i$	j	W
	0	0	0	0	0
	$i-1$	0	$F(i-1,\ j-w_i)$	$F(i-1,\ j)$	
$w_i,\ v_i$	i	0		$F(i,\ j)$	
	n	0			goal

Problema de la mochila

Programación Dinámica: Ejemplo

item	weight	value
1	2	$12
2	1	$10
3	3	$20
4	2	$15

capacity $W = 5$.

F[4,5] = Max { F[3,5] , F[3, 5-2] + 15 }

	capacity j					
i	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	12	12	12	12
2	0	10	12	22	22	22
3	0	10	12	22	30	32
4	0	10	15	25	30	**37**

Problema de la mochila

Memoization (Tomado de la Bibliografía)

ALGORITHM *MFKnapsack*$(i,\ j)$

if $F[i,\ j] < 0$
  if $j < Weights[i]$
    $value \leftarrow MFKnapsack(i-1,\ j)$
  else
    $value \leftarrow \max(MFKnapsack(i-1,\ j),$
      $Values[i] + MFKnapsack(i-1,\ j-Weights[i]))$
  $F[i,\ j] \leftarrow value$
return $F[i,\ j]$

		capacity j					
	i	0	1	2	3	4	5
	0	0	0	0	0	0	0
$w_1 = 2,\ v_1 = 12$	1	0	0	12	12	12	12
$w_2 = 1,\ v_2 = 10$	2	0	—	12	22	—	22
$w_3 = 3,\ v_3 = 20$	3	0	—	—	22	—	32
$w_4 = 2,\ v_4 = 15$	4	0	—	—	—	—	**37**